

EnerVivo

Développement de centrales photovoltaïques

CAHIER DES CHARGES POC

Outil d'audit juridique des projets

Commanditaire : Vincent – EnerVivo / VIVIA

Durée cible : 3 à 5 jours de développement

Budget cible : moins de 20 € de coûts récurrents par mois

Année : 2026

1. Contexte et objectif

EnerVivo doit réunir un grand nombre de pièces juridiques tout au long du développement d'un projet photovoltaïque (Lettre d'Intension, Promesse de Bail et ses pièces attendues en annexe, certificat d'urbanisme, EPA, EIE, PC, CRD, ANRNR, bail notarié, contrats EPC et O&M, etc.). Ces documents sont aujourd'hui stockés dans des sous-dossiers SharePoint, mais avec deux contraintes connues :

- Les fichiers ne sont pas toujours nommés de manière standardisée (le même document peut s'appeler 'Bail.pdf', 'Contrat_signe_v2_def.pdf' ou 'Document final.pdf').
- L'organisation interne des dossiers projet n'est pas homogène : certains documents peuvent être rangés dans des sous-dossiers thématiques, d'autres à la racine.

L'objectif du POC est de vérifier qu'il est possible de :

1. Connecter un outil au SharePoint EnerVivo pour scanner le dossier d'un projet donné.
2. Identifier le type de chaque document par son contenu (et non par son nom de fichier), via Claude API par exemple.
3. Comparer les documents trouvés avec la liste attendue par jalon, pour produire un rapport d'audit listant ce qui est présent, ce qui manque et ce qui est ambigu.

1.1 Rôle du POC

Validation technique	Confirmer la faisabilité de la classification de documents par contenu avec Claude.
Validation fonctionnelle	Vérifier que l'audit produit est utilisable par l'équipe BE et DEV pour identifier rapidement les pièces manquantes d'un projet.
Brique réutilisable	Le pipeline de classification développé ici sera intégré ultérieurement au CRM PPM principal (cf. cahier des charges principal).
Hors périmètre	Pas d'interface CRUD complète, pas de modification du SharePoint, pas de gestion multi-utilisateur, pas de stockage de base de données persistante.

2. Périmètre du POC

2.1 Cas d'usage cible

L'utilisateur (Julien, Antoine ou Pascal) lance le POC sur un projet de la BDD (ex. DDENIS) et obtient en moins de 2 minutes un rapport HTML listant :

- Pour chaque jalon du projet (J1, J2a, J2b, etc.), la liste des documents attendus.
- Pour chaque document attendu, le statut : Trouvé (avec lien SharePoint), Manquant, ou Ambigu (plusieurs candidats).
- Pour les documents trouvés, le nom du fichier réel dans SharePoint et le score de confiance de la classification.

2.2 Périmètre fonctionnel POC

Fonctionnalité	Inclus dans le POC	Hors POC
Connexion SharePoint et lecture de fichiers	Oui	
Extraction texte PDF natifs et DOCX	Oui	
Classification par Claude API	Oui	
Comparaison avec la liste attendue par jalon	Oui	
Rapport HTML imprimable / exportable PDF	Oui	
Audit d'un projet à la fois (CLI ou interface basique)	Oui	
OCR sur PDF scannés		v2 (Tesseract si volumétrie justifie)
Lecture des fichiers Excel et images		v2
Audit en masse de tous les projets		v2
Modification des fichiers SharePoint		Jamais
Interface multi-utilisateur avec authentification		v2 (versions ultérieures)

3. Architecture du POC

3.1 Pipeline de traitement

Le POC est un script exécutable qui prend un code projet en entrée et produit un rapport HTML.

```
[Code projet ex. DDENIS]
|
v
[1] Connexion SharePoint
    Microsoft Graph API authentifié côté application
    |
    v
[2] Listing récursif du dossier projet
    Tous les fichiers (.pdf, .docx) avec leur chemin et leur URL
    |
    v
[3] Extraction texte
    PDF natifs : pdfplumber (Python) ou pdf-parse (Node)
    DOCX : python-docx ou mammoth
    Premiers 3000 caractères + derniers 1000 caractères
    |
    v
[4] Classification par Claude API
    Prompt système : liste des types possibles + instructions
    Prompt utilisateur : extrait du document
    Réponse JSON : { type: ..., confiance: 0-100, raison: ... }
    |
    v
[5] Comparaison avec liste attendue
    documents.config.js (par type de projet et par jalon)
    |
    v
[6] Génération rapport HTML
    Tableau par jalon avec statuts colorés
```

3.2 Stack technique proposée

Composant	Choix POC	Justification
Langage	Python 3.11	Bibliothèques de parsing PDF / DOCX matures, dev rapide, pas de stack à déployer
Connexion SharePoint	msal + Microsoft Graph API	Officiel Microsoft, gratuit, déjà dans la licence M365
Extraction PDF	pdfplumber	Open source, précis sur PDF natifs

Composant	Choix POC	Justification
Extraction DOCX	python-docx	Standard, gratuit
Classification IA	anthropic SDK Python (claude-haiku-4-5)	Modèle rapide et peu coûteux pour cette tâche
Génération rapport	Jinja2 + HTML	Pas de framework lourd, rapport standalone
Exécution	Script CLI (python audit.py DDENIS)	Pas de serveur à maintenir pour le POC

Alternative Node.js possible si le développeur préfère par souci de cohérence avec le futur CRM React. Les bibliothèques équivalentes existent (microsoft-graph-client, pdf-parse, mammoth, anthropic SDK Node).

4. Connexion SharePoint

4.1 Authentification

Le POC utilise une authentification application (App-only) via Microsoft Entra ID (anciennement Azure AD), avec les permissions Sites.Read.All et Files.Read.All. Pas d'authentification utilisateur dans le POC.

1. Créer une App registration dans le tenant Microsoft 365 EnerVivo.
2. Donner les permissions API Sites.Read.All et Files.Read.All (avec accord administrateur).
3. Générer un client secret valable 6 mois pour le POC.
4. Stocker tenant_id, client_id, client_secret dans un fichier .env.

4.2 Localisation du dossier projet

Le code projet (ex. DDENIS) doit pouvoir être converti en chemin SharePoint. Deux stratégies sont possibles selon la convention de nommage des dossiers :

Convention 1 : code projet en préfixe	Le dossier s'appelle DDENIS_Denis_Florian ou DDENIS – recherche directe par préfixe.
Convention 2 : recherche par champ	Une colonne SharePoint custom contient le code projet – utilisation de la search API Graph.
Convention 3 : mapping manuel	Un fichier mapping.csv associe code projet et URL SharePoint – fallback simple si les deux ci-dessus échouent.

Le POC commence par la convention 3 (la plus simple) et évolue vers la convention 1 ou 2 si la recherche automatique est fiable.

5. Classification des documents

5.1 Principe

Pour chaque fichier trouvé dans le dossier projet, le pipeline extrait le texte et envoie un échantillon à Claude avec un prompt spécifique. Claude répond avec le type identifié, un score de confiance, et une explication courte.

5.2 Prompt système utilisé

```
Tu es un expert en analyse de documents juridiques et techniques  
liés aux projets photovoltaïques en France.
```

```
Voici la liste des types de documents possibles :
```

- LOI signée (Lettre d'Intention)
- PDB signée (Promesse de Bail)
- Bail notarié signé
- Certificat d'urbanisme
- Attestation MSA
- APS / APD-A / APD-B / APD-C
- PVSyst (étude productible)
- Rapport EIE / EPA / VNEI / G2AVP
- Avis CDPENAF
- Dépôt PC ou DP / Récépissé PC ou DP
- ANRNR (Attestation de non recours)
- PV de passage huissier
- CRD signée (raccordement ENEDIS)
- Mandat raccordement / Demande raccordement S26
- Kbis SPV / Statuts SPV / Pacte d'actionnaires
- Contrat EPC / AMO / O&M
- Assurance RC Pro / DO / décennale
- Contrat d'achat électricité (OA, AO CRE, ACC, PPA)
- DOE / DAT / CONSUEL
- Compte-rendu RDV (mairie, ComCom, chambre agri, DDT, SDIS)
- Autre / Non identifié

```
Réponds uniquement en JSON avec les clés :
```

```
type: le nom exact d'un type de la liste ci-dessus  
confiance: entier de 0 à 100  
raison: 1 phrase justifiant le choix
```

5.3 Coût de classification

Modèle retenu	claude-haiku-4-5 (rapide et peu coûteux)
Tokens par fichier	Environ 1500 tokens en entrée (extrait + prompt) + 100 tokens en sortie

Coût par fichier	Environ 0,001 € (selon tarif en vigueur)
Audit complet d'un projet	20 à 50 fichiers → 0,02 à 0,05 €
Audit de 100 projets	Environ 5 €
Conclusion	Coût négligeable, pas de risque budgétaire

6. Logique d'audit par jalon

Le POC s'appuie sur le fichier documents.config.js défini dans le cahier des charges principal (section 4.3). Ce fichier liste pour chaque type de projet (AgriPV, S2I) la liste des documents attendus par jalon avec leur critère (Obligatoire / Conditionnel / Informatif).

6.1 Algorithme d'audit

1. Récupération du type de projet (AgriPV ou S2I) depuis la BDD ou paramètre CLI.
2. Récupération des jalons déjà validés pour ce projet (depuis la BDD ou paramètre CLI).
3. Pour chaque jalon validé ou en cours : récupération de la liste de documents attendus depuis documents.config.
4. Pour chaque document attendu : recherche dans la liste des fichiers classifiés du projet.
5. Si trouvé avec confiance $\geq 70\%$: statut Trouvé.
6. Si trouvé avec confiance entre 40% et 70% : statut Ambigu (revue manuelle).
7. Si non trouvé ou confiance $< 40\%$: statut Manquant.

6.2 Cas particuliers à gérer

- Plusieurs fichiers identifiés comme le même type (ex. 3 PV de passage huissier) : tous listés, statut Trouvé multiple.
- Document conditionnel (ex. attestation MSA) : si le projet n'est pas AgriPV, marqué Non applicable et non comptabilisé dans les manquants.
- Fichier non classifiable (ex. photo, fichier vide, document très spécifique non listé) : marqué Autre, listé dans une section séparée du rapport pour revue manuelle.
- Fichier protégé par mot de passe ou corrompu : marqué Erreur lecture, listé dans la section technique du rapport.

7. Restitution

7.1 Mode d'exécution

Le POC est exécuté en ligne de commande pour la simplicité :

```
python audit.py --projet DDENIS --jalons J1,J2a,J2b
# Génère le fichier audit_DDENIS_2025-04-27.html
```

Une interface web minimale (formulaire avec champ code projet) peut être ajoutée si le temps le permet, hébergée sur Vercel gratuit.

7.2 Contenu du rapport HTML

Entête	Code projet, nom client, type projet, date d'audit, nombre de fichiers scannés
Résumé exécutif	Pourcentage de complétude global / par jalon. Top 3 documents critiques manquants.
Tableau par jalon	Liste des documents attendus avec statut coloré, lien fichier, score confiance, fichier source
Section Documents non identifiés	Fichiers du dossier non rattachés à un type connu – à vérifier manuellement
Section Erreurs techniques	Fichiers protégés ou illisibles – nom et chemin SharePoint
Section Métadonnées	Liste exhaustive des fichiers scannés avec horodatage et taille

7.3 Codes couleurs du rapport

Statut	Couleur	Signification
Trouvé	Vert	Document identifié avec confiance ≥ 70 %
Ambigu	Orange	Document trouvé mais confiance < 70 % – revue manuelle conseillée
Manquant	Rouge	Aucun fichier identifié comme ce type
Non applicable	Gris	Document conditionnel non requis pour ce type de projet

8. Plan de réalisation

Jour	Activité	Livrable
J1 matin	Setup environnement Python, App registration Azure, test connexion Graph API	Connexion SharePoint fonctionnelle, fichier .env configuré
J1 après-midi	Fonction de listing récursif des fichiers d'un dossier projet	list_project_files(code_projet) retourne la liste des chemins
J2 matin	Fonctions d'extraction texte PDF (pdfplumber) et DOCX (python-docx)	extract_text(file_path) retourne 4000 caractères significatifs
J2 après-midi	Codage du fichier documents.config.json (liste complète par jalon AgriPV et S21)	documents.config.json validé par DEV et BE
J3 matin	Intégration Claude API et fonction de classification	classify_document(text) retourne JSON {type, confiance, raison}
J3 après-midi	Logique d'audit : comparaison documents trouvés et liste attendue par jalon	audit_project(code_projet) retourne un dict structuré
J4 matin	Génération du rapport HTML avec Jinja2 et codes couleur	Rapport HTML standalone avec CSS inline
J4 après-midi	Tests sur 5 projets réels EnerVivo (DDENIS, DDOMINE, DFRAISSE, etc.)	5 rapports d'audit générés, validation par Vincent
J5	Corrections et ajustements selon retour utilisateur	Version 1.0 stable, documentation README, livraison

9. Coûts du POC

Poste	Coût initial	Récurrent mensuel
Développement (5 jours)	Selon TJM équipe interne ou prestataire	0
Microsoft Graph API	0 € (inclus dans M365 EnerVivo)	0 €
Claude API	Pay per use	Environ 5 € (estimé 100 audits/mois)
Hosting	0 € (script local ou Vercel gratuit si interface web)	0 €
Stockage	0 € (fichiers restent sur SharePoint)	0 €
Total	TJM équipe x 5 jours	Moins de 10 €/mois

10. Risques et limites

Risque ou limite	Impact	Mitigation
PDF scannés non lisibles sans OCR	Moyen	Le POC ignore les PDF sans texte natif. Si volumétrie significative, ajout d'OCR Tesseract en v2.
Confiance Claude trop basse sur certains types	Moyen	Ajustement du prompt avec exemples concrets. Tests sur 5 projets pour calibrer le seuil 70 %.
Convention de nommage SharePoint hétérogène	Moyen	Fallback sur fichier mapping.csv code projet vers URL SharePoint.
Documents protégés par mot de passe	Faible	Levés en erreur dans le rapport, à vérifier manuellement.
Coût API Claude qui dérape	Faible	Modèle Haiku peu coûteux, alerte budget Anthropic configurée à 50 €/mois.
Mauvaise classification de documents proches	Moyen	Score confiance et statut Ambigu pour forçage manuel. Apprentissage du prompt par itérations.

11. Évolutions vers la production

Une fois le POC validé, les évolutions suivantes sont envisageables, par ordre de priorité :

1. Intégration au CRM PPM principal : la classification est appelée automatiquement quand un fichier est uploadé ou rattaché à un projet, et le statut du document attendu est mis à jour automatiquement.
2. Audit en masse : un planificateur passe en revue tous les projets actifs chaque semaine et génère un rapport global pour la Direction.
3. OCR pour PDF scannés : ajout de Tesseract si la volumétrie justifie.
4. Lecture des fichiers Excel et images (avec OCR ou Claude Vision).
5. Interface web complète avec authentification, historique des audits, comparaison entre deux dates.
6. Notifications Make.com sur les documents critiques manquants à J-15 d'un jalon.

12. Questions à trancher avant le démarrage

Questions structurelles

- Quelle est la convention de nommage actuelle des dossiers projet sur SharePoint ? Code projet en préfixe, nom client, autre ?
- Existe-t-il une structure de sous-dossiers thématiques (ex. /01_Foncier, /02_Permitting, /03_Raccordement) ou tout est-il à la racine ?
- Quel pourcentage estimé de PDF sont des scans (sans texte natif) versus des PDF natifs générés ? Cela conditionne l'urgence de l'OCR.

Questions techniques

- Qui dans l'équipe IT a les droits administrateur Microsoft 365 pour créer l'App registration et accorder les permissions ?
- Le POC est-il développé en Python (recommandé pour rapidité) ou en Node.js (cohérence avec le futur CRM) ?
- Préférence pour une CLI simple ou une interface web minimale dès la v1 du POC ?

Questions fonctionnelles

- Quel est le seuil de confiance acceptable pour valider une classification ? 70 % est une piste, mais peut être ajusté.
- Le POC doit-il inclure d'emblée les projets S2I ou seulement AgriPV pour la phase de validation ?
- Sur quels projets EnerVivo voulez-vous tester en priorité ? 5 projets représentatifs sont recommandés.